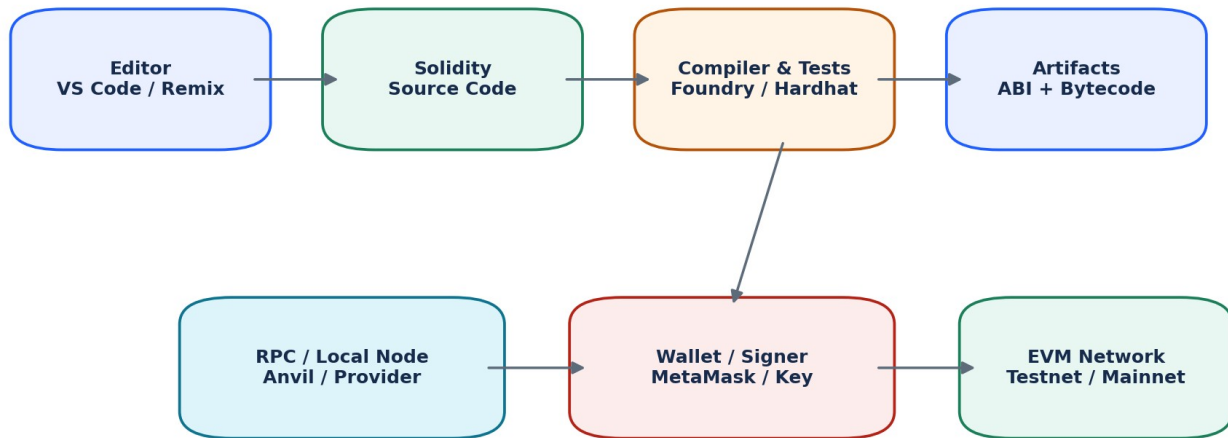


DAY 4

Professional Blockchain Development Environment

Beginner -> Job-Ready Workflow for Solidity / EVM Freelancing

COURSE GOAL Set up and understand the professional toolchain used to write, compile, test, debug, deploy, verify, and maintain EVM smart-contract projects.



Professional workflow: write -> compile -> test -> deploy -> verify -> monitor

Lecture	Duration	Primary Outcome
Day 4	60 minutes	A working mental model and repeatable local blockchain development workflow
Level	Beginner	No production funds or production private keys required
Career focus	LinkedIn + Upwork	Deliver reproducible projects, not one-off code snippets

SAFETY RULE Use only test accounts and test funds during this lecture. Never type a real seed phrase or production private key into examples, repositories, screenshots, issue trackers, or chat.

1. Learning Objectives & 60-Minute Schedule

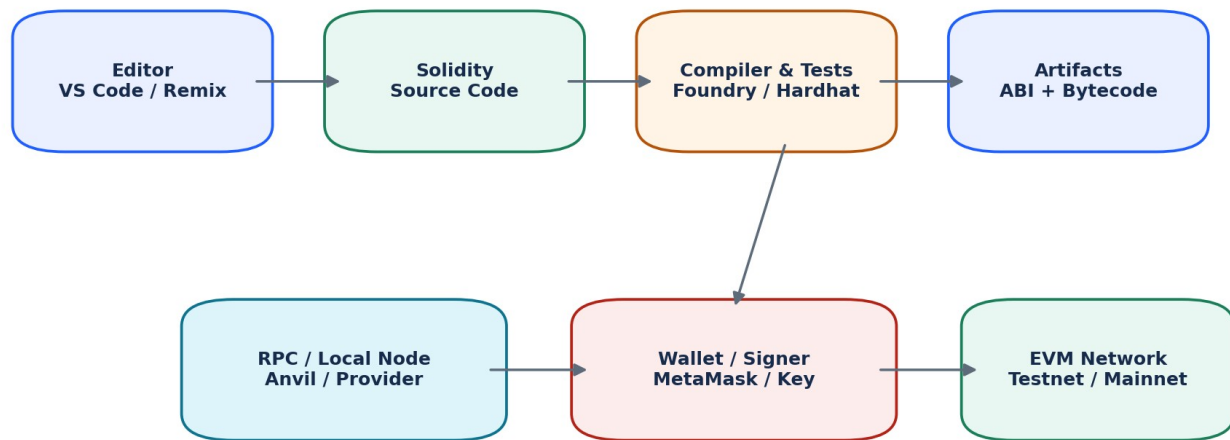
By the end of this lecture, students should be able to explain the purpose of each major development tool and create a clean project workflow from source code to a local or testnet deployment.

- Explain the roles of VS Code, Remix, Git, Node.js, npm/pnpm, Foundry, Hardhat, ethers.js, MetaMask, RPC endpoints, and explorers.
- Choose when to use Remix versus a local project.
- Verify that required command-line tools are installed and available in PATH.
- Understand compile artifacts: ABI, bytecode, metadata, build/cache folders.
- Run a local blockchain and separate local, testnet, and mainnet configuration.
- Use environment variables and .gitignore to keep secrets out of source control.
- Follow a freelancer-quality workflow: branch -> code -> test -> deploy -> verify -> document -> commit.

Minutes	Topic	Teaching Outcome
0-5	The blockchain developer toolchain	See how all tools fit together.
5-12	Terminal, Git, Node.js, package managers	Prepare the base software environment.
12-20	Remix IDE	Compile/deploy a first contract with zero local setup.
20-34	Foundry & Hardhat	Understand project structures, testing, scripts, local nodes.
34-43	RPC, wallets, networks, environment variables	Connect software safely to an EVM network.
43-51	ethers.js + first read call	See how JavaScript talks to Ethereum.
51-57	Hands-on project setup	Create/verify a clean dev project.
57-60	Freelancer checklist, quiz, homework	Turn tooling into a client-ready workflow.

INSTRUCTOR PROMPT Ask: "If Solidity is the language, what other tools do we need before a client can actually use the contract?" Use answers to introduce the full toolchain.

2. The Professional Toolchain Mental Model | 0-5 min



Professional workflow: write -> compile -> test -> deploy -> verify -> monitor

Figure 1. A modern EVM development toolchain from source code to network deployment.

A blockchain developer is not only a Solidity programmer. A professional project normally combines source control, compilers, test runners, RPC access, signing, deployment scripts, frontend/backend libraries, and documentation.

Layer	Typical Tool	Why It Exists
Editor / IDE	VS Code, Remix	Write code, inspect files, run extensions, navigate errors.
Version control	Git	Track changes, branches, commits, code reviews, rollback.
Runtime / packages	Node.js + npm/pnpm	Run JavaScript tooling and install dependencies.
Smart-contract framework	Foundry / Hardhat	Compile, test, debug, deploy, verify.
Wallet / signer	MetaMask / local key / hardware wallet	Authorize state-changing transactions.
RPC	Local node or provider endpoint	Read chain data and broadcast signed transactions.
Client library	ethers.js	Create providers, contracts, signers, calls, transactions.
Explorer	Etherscan-like explorer	Inspect transactions, contracts, events, verification status.

FREELANCER HABIT When a client gives you a repository, first identify its framework, package manager, compiler version, network configuration, environment variables, and existing test commands before changing code.

3. Terminal, Git, Node.js & Package Managers | 5-12 min

3.1 The Terminal Is Part of the Job

Even when an IDE has buttons, professional blockchain projects rely heavily on terminal commands. Learn to read command output and error messages; do not treat the terminal as a black box.

```
pwd          # show current directory
ls           # list files (macOS/Linux)
dir          # list files (Windows cmd)
cd project   # enter a directory
mkdir project # create a directory
```

3.2 Git: Reproducibility and Client Safety

```
git --version
git status
git branch
git log --oneline -5
```

Git is not only for backup. It provides a history of what changed, who changed it, and which version was delivered to a client.

CLIENT RULE Before modifying an unfamiliar repository, create or use a branch unless the client explicitly requires a different workflow. Never “experiment” directly on a production deployment branch.

3.3 Node.js and npm/pnpm

Node.js runs JavaScript outside the browser. Many blockchain tools, frontend apps, deployment scripts, and libraries use the Node ecosystem. npm ships with normal Node installations; pnpm or Yarn may be used by a repository instead.

```
node --version
npm --version
# If the repository has pnpm-lock.yaml, also check:
pnpm --version
```

File	Meaning
package.json	Project metadata, scripts, dependency declarations.
package-lock.json	npm dependency lockfile.
pnpm-lock.yaml	pnpm dependency lockfile.
node_modules/	Installed packages; normally not committed.
README.md	Human setup and usage instructions.

4. Remix IDE: Fast Solidity Learning & Prototyping | 12-20

min

Remix is a browser-based Ethereum IDE. It is excellent for beginners, fast experiments, contract inspection, and small reproductions because you can create, compile, deploy, and interact with Solidity without first creating a local JavaScript project.

4.1 Main Areas to Teach

Area	What Students Should Learn
File Explorer	Create .sol files and organize a workspace.
Solidity Compiler	Choose the compiler compatible with the contract and compile.
Deploy & Run	Choose an execution environment and deploy/interact.
Terminal / Console	Read transaction output, warnings, and errors.
Debugger / Analysis	Investigate failed calls and suspicious patterns.

4.2 First Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract Counter {
    uint256 public value;

    function increment() external {
        value += 1;
    }
}
```

1. Create Counter.sol in Remix.
2. Compile the contract.
3. Deploy to the in-browser/local Remix VM first.
4. Read value(). It should return 0.
5. Call increment(). A transaction is created.
6. Read value() again. It should return 1.

TEACHING POINT A read-only call does not need to change blockchain state, while increment() changes state and therefore requires a transaction in a real network environment.

WHEN NOT TO STOP AT REMIX For serious client work you still need reproducible local builds, automated tests, deployment scripts, source control, CI, and dependency management. That is where Foundry/Hardhat become important.

5. Foundry & Hardhat: Local Professional Frameworks | 20-34 min

Foundry and Hardhat solve overlapping problems: compilation, testing, debugging, scripts, deployment, local development networks, and project configuration. You do not need to hate one to use the other. A freelancer should be able to enter either ecosystem.

Capability	Foundry	Hardhat
Primary workflow	CLI + Solidity-native tooling	Node/TypeScript ecosystem with plugins
Compile	forge build	Hardhat compile task
Tests	Solidity tests, fuzzing, invariants	Solidity and/or TypeScript testing depending on setup
Local node	anvil	Hardhat local network / node
Chain interaction	cast	Scripts + ethers/plugins
Best beginner use	Fast smart-contract focus	Full-stack JS/TS project integration

CURRENT TOOLING NOTE Hardhat 3 is the current major Hardhat generation, while Foundry provides Forge, Cast, Anvil, and Chisel. Always read the repository's own package/config files before assuming commands.



If a step fails, debug before moving forward. Do not “deploy and hope.”

Figure 2. The development loop you should repeat on every smart-contract task.

5.1 Foundry Verification Commands

```
forge --version
cast --version
anvil --version
```

5.2 Typical Foundry Project

```
forge init my-project
cd my-project
forge build
forge test
```

IMPORTANT The exact installer command can evolve. Install Foundry through the official Foundry documentation, then verify forge, cast, and anvil from your terminal.

6. Foundry Project Anatomy & Local Chain

Path	Purpose
src/	Smart contracts.
test/	Solidity tests.
script/	Deployment / interaction scripts.
lib/	Dependencies.
foundry.toml	Foundry project configuration.
out/	Compiler artifacts; generated.
cache/	Build cache; generated.

6.1 Build and Test

```
forge build
forge test
forge test -vvv
```

Verbosity flags are useful when you need more information about a failure. Later lectures will cover unit tests, fuzzing, traces, and security testing in depth.

6.2 Local Blockchain with Anvil

```
anvil
```

A local node gives you fast blocks, deterministic test accounts, and no real-money risk. It is ideal for development and debugging.

SAFETY The private keys printed by a local development node are development-only keys. Never reuse them on a real funded network.

6.3 Cast: Talk to an EVM Chain from the Terminal

```
cast block-number --rpc-url http://127.0.0.1:8545
cast balance <ADDRESS> --rpc-url http://127.0.0.1:8545
```

Cast is useful for quick RPC queries, calldata encoding/decoding, receipts, storage inspection, and transaction debugging. You will use these skills later when diagnosing client issues.

7. Hardhat Project Anatomy

Hardhat is especially common in JavaScript/TypeScript teams because smart-contract work can live in the same Node ecosystem as deployment scripts, backend services, and frontend tooling.

7.1 Start with a Normal Node Project

```
mkdir hardhat-demo
cd hardhat-demo
npm init -y
npm install --save-dev hardhat
npx hardhat --init
```

The initialization workflow may prompt you to select a project type and dependencies. Follow the current Hardhat documentation and the repository's existing conventions.

7.2 Key Files You May Encounter

Path	Purpose
hardhat.config.*	Network/compiler/plugin configuration.
contracts/	Solidity contracts in many templates.
test/	Automated tests.
scripts/ or ignition/	Deployment and automation, depending on project style.
artifacts/	Generated ABI/bytecode metadata.
cache/	Generated compilation cache.

7.3 npm Scripts Are Client Documentation

```
{
  "scripts": {
    "compile": "hardhat compile",
    "test": "hardhat test"
  }
}

npm run compile
npm test
```

FREELANCER TIP Before installing or upgrading dependencies, inspect package.json and the lockfile. Blindly updating framework/plugin versions can break a working client repository.

8. Compile Artifacts: ABI, Bytecode & Metadata

Compilation transforms Solidity source into artifacts used by deployment tools and applications. The two terms you must understand today are ABI and bytecode.

Artifact	What It Is	Who Uses It
ABI	Machine-readable description of callable functions/events/errors.	Frontend, backend, scripts, explorers.
Creation bytecode	Code used when deploying a contract.	Deployment transaction.
Runtime bytecode	Code stored at the deployed contract address.	EVM during contract calls.
Metadata	Compiler/source/config information.	Verification and tooling.

8.1 Example ABI Fragment

```
[
  {
    "type": "function",
    "name": "value",
    "stateMutability": "view",
    "inputs": [],
    "outputs": [{"type": "uint256"}]
  }
]
```

COMMON CLIENT PROBLEM “The frontend cannot call my contract” often turns into an ABI/address/network mismatch. Always check all three before rewriting code.

9. RPC, Wallets, Chain IDs & Networks | 34-43 min

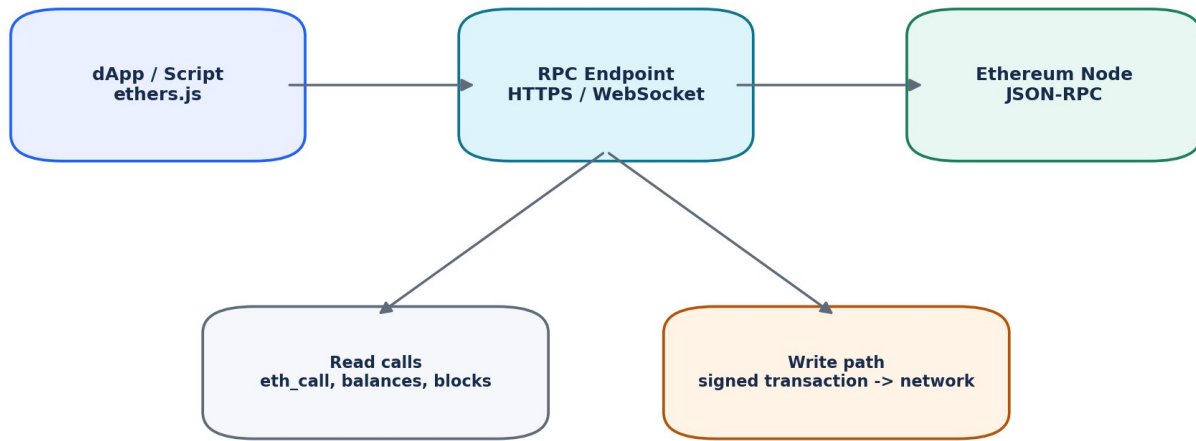


Figure 3. JavaScript applications normally reach an EVM chain through a JSON-RPC endpoint.

9.1 What Is an RPC Endpoint?

An RPC endpoint is an interface to a blockchain node. Your application sends requests such as “get block number,” “read contract state,” or “broadcast this signed transaction.”

```
https://example-rpc-provider/...
ws://example-websocket-provider/...
```

9.2 Read vs Write

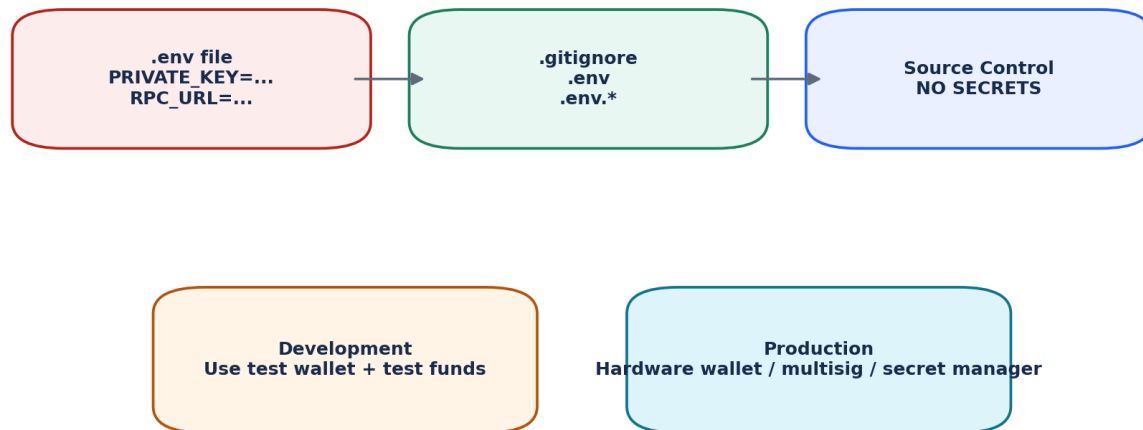
Operation	Needs Private Key?	Example
Read public state	No	Get balance, block, contract view function.
Simulate a call	No signer required	eth_call / estimate gas.
Send state-changing transaction	Yes, some signer must authorize	Transfer token, call stake(), deploy contract.

9.3 Network Identity

A chain ID distinguishes networks and helps protect signed transactions from being replayed on another chain. A contract address is meaningful only together with its network.

DEBUG RULE When a client says “the contract is missing,” ask for network + chain ID + address. The same hexadecimal address can represent unrelated accounts/contracts on different chains.

10. Environment Variables & Secret Management



Never paste a real seed phrase or private key into code, chat, issue trackers, or repositories.

Figure 4. Keep local secrets outside source control and separate development from production signing.

10.1 Local .env Pattern

```
RPC_URL=https://...
PRIVATE_KEY=0xTEST_ONLY_PRIVATE_KEY
EXPLORER_API_KEY=...
```

10.2 .gitignore

```
node_modules/
.env
.env.*
!/.env.example
```

10.3 Commit an Example, Not the Secret

```
# .env.example
RPC_URL=
PRIVATE_KEY=
EXPLORER_API_KEY=
```

PRODUCTION PRACTICE For serious deployments, prefer hardware wallets, multisig workflows, CI/CD secret stores, or managed secret systems over a long-lived production private key in a plain local file.

10.4 What To Do if a Secret Is Committed

7. Treat the secret as exposed, even if the repository was private.
8. Rotate/revoke the affected API key or move assets/permissions from the exposed signing key.
9. Remove the secret from current files and prevent recurrence with ignore rules and scanning.
10. Coordinate history rewriting only when appropriate; rewriting Git history does not make an already-exposed key safe again.

11. ethers.js: First Programmatic Blockchain Read | 43-51

min

ethers.js is a JavaScript/TypeScript library for interacting with Ethereum-compatible networks. It provides providers, wallets/signers, contracts, transaction utilities, ABI tools, and unit conversion helpers.

11.1 Install

```
npm install ethers
```

11.2 Provider Example

```
import { JsonRpcProvider, formatEther } from "ethers";

const provider = new JsonRpcProvider(process.env.RPC_URL);

const blockNumber = await provider.getBlockNumber();
console.log("block:", blockNumber);

const balance = await provider.getBalance("0x0000000000000000000000000000000000000000");
console.log("balance:", formatEther(balance));
```

This example performs read operations only. No private key is required because reading public blockchain state does not need transaction authorization.

11.3 Contract Read

```
import { Contract } from "ethers";

const abi = ["function balanceOf(address) view returns (uint256)"];
const token = new Contract(TOKEN_ADDRESS, abi, provider);
const balance = await token.balanceOf(USER_ADDRESS);
```

CORE MODEL Provider = connection to blockchain data. Signer = authority to sign. Contract = address + ABI + provider/signer.

12. Hands-On Lab: Build a Clean Local Workspace | 51-57 min

The goal is not to install every possible tool. The goal is to prove that your environment is understandable, reproducible, and safe.

Lab A - Environment Health Check

```
git --version
node --version
npm --version
forge --version
cast --version
```

If a command is “not found,” do not randomly reinstall everything. Identify which tool is missing and whether your PATH or terminal session needs to be refreshed.

Lab B - Create a Workspace

```
mkdir blockchain-day4
cd blockchain-day4
git init
npm init -y
npm install ethers
```

Lab C - Minimal Git Hygiene

```
echo node_modules/ >> .gitignore
echo .env >> .gitignore
git add .
git status
```

Lab D - Local Chain

```
# Terminal 1
anvil

# Terminal 2
cast block-number --rpc-url http://127.0.0.1:8545
```

SUCCESS CRITERIA You can explain what each command does, the local RPC responds, generated/private files are ignored, and git status contains only files you intentionally want to commit.

13. Client-Ready Project Structure

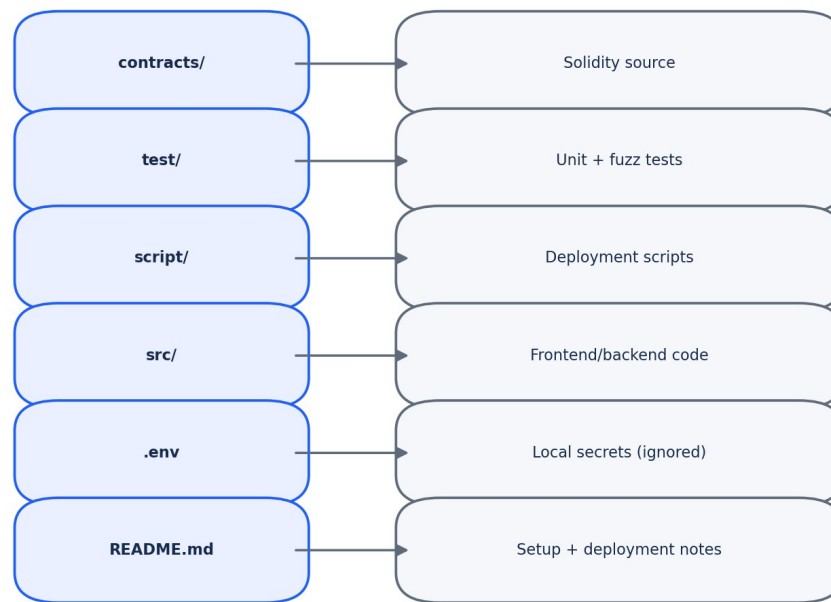


Figure 5. A clean project separates contracts, tests, scripts, application code, secrets, and documentation.

13.1 README Minimum

- Project purpose and scope.
- Prerequisites and tool versions/ranges where important.
- Installation command.
- Compile command.
- Test command.
- Local-node instructions.
- Required environment variables (names only, not secrets).
- Deployment instructions and deployed testnet addresses.
- Known limitations and security assumptions.

13.2 Why This Wins Outsourcing Jobs

A client is buying maintainability, predictability, and reduced risk - not only Solidity syntax. Clean setup instructions and repeatable tests make it easier for another developer to review, run, and continue your work.

PORTFOLIO IDEA Publish a tiny repository called `evm-dev-environment-demo` containing a Counter contract, one test, one deployment script, `.env.example`, `.gitignore`, and a clear README. This demonstrates professional hygiene before your larger DeFi projects.

14. Common Environment Problems & How to Think

Symptom	Likely Cause	First Checks
command not found	Tool not installed or PATH not refreshed.	Verify install location; restart terminal; inspect PATH.
npm script missing	Wrong folder or package.json lacks script.	pwd/cd; open package.json; npm run.
module not found	Dependencies not installed or import/package mismatch.	Install from lockfile; inspect package.json.
compiler version error	Contract pragma/config mismatch.	Read pragma and framework compiler config.
RPC connection fails	Wrong URL, outage, auth issue, firewall.	Check URL, network, provider status, curl/request.
insufficient funds	Signer lacks native gas token.	Check signer address, network, native balance.
wrong network	Wallet/RPC chain differs from deployment.	Check chain ID and contract address.
nonce too low / pending tx	Conflicting/pending transactions.	Inspect account nonce and pending queue.
frontend method missing	Wrong ABI or address.	Compare deployed contract + ABI + network.
secret leaked in Git	Unsafe env handling.	Rotate first; then clean repository and add protections.

14.1 Debugging Order

11. Reproduce the exact command and capture the first meaningful error.
12. Confirm you are in the correct repository and branch.
13. Check runtime/tool versions and package manager.
14. Install dependencies from the existing lockfile.
15. Compile before deploying.
16. Run the smallest failing test or script.
17. Verify network, chain ID, RPC, signer, balances, and addresses.
18. Only then change code or dependencies.

PROFESSIONAL BEHAVIOR Do not hide build errors from a client. Report the exact failing step, root cause if known, and the smallest safe fix.

15. Upwork / LinkedIn Client Scenarios

Scenario A - “Hardhat project does not run on my machine”

Your response process: inspect README and package.json, determine Node/package-manager expectations, install from the lockfile, run compile/test, record the first error, and avoid dependency upgrades until you know why the project fails.

Scenario B - “Deploy this contract to BSC/Base/Arbitrum”

Before deployment you need: target chain ID, RPC endpoint, funded test/production signer, constructor arguments, compiler settings, deployment script, verification plan, ownership/admin requirements, and a rollback/hand-off plan.

Scenario C - “Frontend says contract method is undefined”

Check whether the frontend uses the ABI from the deployed contract version, whether the contract address is correct, and whether the wallet/provider is on the same network.

Scenario D - “Can I send you my private key?”

A professional default is to avoid receiving a client's production private key. Prefer test deployments, multisig/hardware-wallet signing, or a client-controlled signing step when possible.

CLIENT QUESTION TEMPLATE “Please provide the repository/commit, target network, deployed address (if any), exact failing command or transaction hash, and sanitized environment-variable names. Do not send seed phrases or production private keys.”

16. Interview Questions + 10-Question Quiz | 57-60 min

Interview Questions

Question	Strong Short Answer
Why use Git?	To version source, review changes, reproduce releases, and safely collaborate.
Remix vs Foundry/Hardhat?	Remix is great for fast browser prototyping; local frameworks add reproducible builds, automated tests, scripts, dependencies, and team workflows.
What is an RPC URL?	An endpoint through which software sends JSON-RPC requests to a blockchain node.
Provider vs signer?	Provider reads/connects to chain data; signer can authorize transactions/messages.
What is an ABI?	The interface description used to encode/decode contract functions, events, errors, and values.
Why .env + .gitignore?	To keep environment-specific secrets/configuration outside committed source code.
Why local node?	Fast deterministic testing without real funds or public-network latency.
What should you verify before deployment?	Tests, compiler/settings, network/chain ID, signer balance, constructor args, admin ownership, and verification process.

Mini Quiz

19. What are the three most important files to inspect when entering a Node-based blockchain repository?
20. What does forge build do?
21. What is Anvil used for?
22. Does reading a public contract view function require a private key?
23. What three pieces are commonly needed to instantiate a contract client?
24. Why should generated node_modules not normally be committed?
25. What is the danger of blindly upgrading dependencies in a client project?
26. If the same contract address is shown on two networks, are they automatically the same contract?
27. What should you do first if a production private key was accidentally committed?
28. Which environment should a beginner deploy to first: local/testnet or mainnet? Why?

17. Homework: Day 4 Environment Portfolio Task

Task 1 - Tool Health Report

```
git --version
node --version
npm --version
forge --version
cast --version
```

Save the output in your notes. Do not include secrets or private keys.

Task 2 - Remix Counter

Compile and deploy the Counter contract in a safe Remix development environment. Record: compiler version, deployment environment, contract address in that environment, one increment transaction, and the final value.

Task 3 - Local Foundry Project

```
forge init day4-counter
cd day4-counter
forge build
forge test
```

Read the generated folders and identify src, test, script, lib, out, and foundry.toml.

Task 4 - Professional Repository Hygiene

- Create .gitignore.
- Create .env.example with empty values.
- Write a README with setup/build/test steps.
- Commit only source/config/documentation that belongs in the repository.
- Run git status before every commit.

Task 5 - Explain the Workflow in 90 Seconds

Be able to say: "I write Solidity, compile it into ABI/bytecode, run automated tests, deploy locally, deploy to a testnet through an RPC using a signer, verify the contract, and commit the reproducible project to Git."

18. Day 4 Printable Cheat Sheet

Tool / Concept	Remember This
VS Code	Local editor and development workspace.
Remix	Browser IDE for fast Solidity compile/deploy/interact learning.
Git	Source history, branches, collaboration, rollback.
Node.js	JavaScript runtime used by many blockchain tools.
npm / pnpm	Package managers; follow the repository lockfile.
Foundry	Forge + Cast + Anvil + Chisel toolchain.
Hardhat	Node/TypeScript-centered Ethereum development environment.
Forge	Build, test, scripts/deployment in Foundry.
Anvil	Local EVM development node.
Cast	CLI for EVM/RPC interaction.
ethers.js	JavaScript/TypeScript Ethereum client library.
RPC URL	Connection endpoint to a blockchain node.
Provider	Read/connect to network.
Signer	Authorizes messages/transactions.
ABI	Contract interface for encoding/decoding calls/events.
Bytecode	EVM-executable contract code.
.env	Local configuration/secrets - do not commit.
.gitignore	Prevents specified local/generated files from being committed.
Chain ID	Network identifier used by transaction/network logic.
Explorer	Inspect and verify deployed contracts/transactions.

DAY 4 COMPLETION STANDARD You are ready for Day 5 when you can explain the whole toolchain without notes and can create, compile, test, and inspect a small development project without using real production keys or funds.

19. Official References & Next Lecture

Use official documentation as the source of truth for installation commands and current tool behavior. Blockchain frameworks evolve quickly, so do not rely on an old tutorial when the project's official docs disagree.

Resource	Official Documentation
Hardhat	https://hardhat.org/docs/getting-started
Foundry	https://getfoundry.sh/
Remix IDE	https://remix-ide.readthedocs.io/
Node.js	https://nodejs.org/en/download
Git	https://git-scm.com/book/en/v2/Getting-Started-Installing-Git
ethers.js v6	https://docs.ethers.org/v6/getting-started/
MetaMask Developer Docs	https://docs.metamask.io/
Solidity Docs	https://docs.soliditylang.org/

Next: Day 5 - Solidity Fundamentals I

Day 5 moves from environment setup into coding: contract structure, pragma/SPDX, state variables, value/reference types, functions, visibility, view/pure/payable, arrays, mappings, memory/storage/calldata, compilation, deployment, and a beginner mini-project.

PREP FOR DAY 5 Keep Remix available and make sure at least one local framework works. Day 5 should be spent learning Solidity concepts, not fighting an unverified environment.